

Assignment-9.5

K.Nikshitha

2303A51692

Batch-24

Problem 1: String Utilities Function

Consider the following Python function:

```
def reverse_string(text):
```

```
    return text[::-1]
```

Task:

1. Write documentation in:

o (a) Docstring

o (b) Inline comments

o (c) Google-style documentation

2. Compare the three documentation styles.

3. Recommend the most suitable style for a utility-based string library.

```
1  #Docstring
2  def reverse_string(text):
3      """
4      This function takes a string as input and returns the reversed version of that string.
5      """
6
7      return text[::-1]
8  print(reverse_string.__doc__)
9
10 #google style documentation
11 def reverse_string(text):
12     """
13     Reverses the input string.
14
15     Args:
16         text (str): The string to be reversed.
17
18     Returns:
19         str: The reversed string.
20     """
21     return text[::-1]
22 print(reverse_string.__doc__)
23
24 #Inline comments
25 def reverse_string(text):
26     # Use slicing with a step of -1 to reverse the string
27     return text[::-1]
28 print(reverse_string.__doc__) #This will print None because there is no docstring defined for the function.
29
```

```
This function takes a string as input and returns the reversed version of that string.
```

```
Reverses the input string.
```

```
Args:  
    text (str): The string to be reversed.
```

```
Returns:  
    str: The reversed string.
```

```
This function takes a string as input and returns the reversed version of that string.
```

```
Reverses the input string.
```

```
Args:  
    text (str): The string to be reversed.
```

```
Reverses the input string.
```

```
Args:  
    text (str): The string to be reversed.  
Reverses the input string.
```

```
Args:  
    text (str): The string to be reversed.
```

```
Args:  
    text (str): The string to be reversed.  
    text (str): The string to be reversed.
```

```
Returns:  
    str: The reversed string.
```

```
None
```

Problem 2: Password Strength Checker

Consider the function:

```
def check_strength(password):  
    return len(password) >= 8
```

Task:

1. Document the function using docstring, inline comments, and Google style.
2. Compare documentation styles for security-related code.
3. Recommend the most appropriate style.

```

1  #Docstring
2  def check_strength(password):
3      """Checks if password length is at least 8 characters.
4      """
5      return len(password) >= 8
6  print(check_strength.__doc__)
7
8  #Inline comments
9  def check_strength(password):
10     # Password must be at least 8 characters long
11     return len(password) >= 8
12 print(check_strength.__doc__) #This will print None because there is no docstring defined for the function.
13
14 #google style documentation
15 def check_strength(password):
16     """
17     Checks if the password is strong based on its length.
18
19     Args:
20         password (str): The password to be checked.
21     Returns:
22         bool: True if the password is strong (length >= 8), False otherwise.
23     """
24     return len(password) >= 8
25 print(check_strength.__doc__)

```

```

Checks if password length is at least 8 characters.

None

Checks if the password is strong based on its length.

Args:
    password (str): The password to be checked.
Returns:
    bool: True if the password is strong (length >= 8), False otherwise.
PS C:\Users\souky\AppData\Local\Programs\Microsoft VS Code>

```

Problem 3: Math Utilities Module

Task:

1. Create a module `math_utils.py` with functions:
 - o `square(n)`
 - o `cube(n)`
 - o `factorial(n)`
2. Generate docstrings automatically using AI tools.
3. Export documentation as an HTML file.

```

1  def square(x):
2      """Returns the square of a number.
3      Parameters:
4      x (int or float): The number to be squared.
5      Returns:
6      int or float: The square of the input number.
7      """
8      return x * x
9  def cube(x):
10     """Returns the cube of a number.
11     Parameters:
12     x (int or float): The number to be cubed.
13     Returns:
14     int or float: The cube of the input number.
15     """
16     return x * x * x
17  def factorial(n):
18     """Returns the factorial of a number.
19     Parameters:
20     n (int): The number to calculate the factorial of. Must be a non-negative integer.
21     Returns:
22     int: The factorial of the input number.
23     """
24     if n == 0 or n == 1:
25         return 1
26     else:
27         return n * factorial(n - 1)
28  print(square.__doc__)
29  print(cube.__doc__)
30  print(factorial.__doc__)

```

```

Returns the square of a number.
Parameters:
Returns the square of a number.
Parameters:
x (int or float): The number to be squared.
Parameters:
x (int or float): The number to be squared.
x (int or float): The number to be squared.
Returns:
int or float: The square of the input number.

Returns the cube of a number.
Parameters:
x (int or float): The number to be cubed.
Returns:
int or float: The cube of the input number.

Returns the factorial of a number.
Parameters:
n (int): The number to calculate the factorial of. Must be a non-negative integer.
Returns:
int: The factorial of the input number.

```

PS C:\Users\couda\AppData\Local\Programs\Microsoft VS Code>

Problem 4: Attendance Management Module

Task:

1. Create a module attendance.py with functions:
 - o mark_present(student)
 - o mark_absent(student)
 - o get_attendance(student)
2. Add proper docstrings.
3. Generate and view documentation in terminal and browse

```

1 attendance = { }
2 def mark_present(student):
3     """
4     marks a student as present in the attendance record
5     parameters:
6     student (str): the name of the student to mark as present
7     """
8     attendance[student] = 'Present'
9
10 def mark_absent(student):
11     """marks a student as absent in the attendance record
12     parameters:
13     student (str): the name of the student to mark as absent
14     """
15     attendance[student] = 'Absent'
16
17 def get_attendance(student):
18     """Returns the attendance status of a student.
19     parameters:
20     student (str): the name of the student whose attendance is to be retrieved
21     returns:
22     str: the attendance status of the student (Present or Absent)
23     """
24     return attendance.get(student, 'Student not found')
25 import attendance
26 help(attendance)

```

Help on module attendance:

NAME

attendance

FUNCTIONS

get_attendance(student)

Returns the attendance status of a student.

parameters:

student (str): the name of the student whose attendance is to be retrieved

returns:

str: the attendance status of the student (Present or Absent)

mark_absent(student)

marks a student as absent in the attendance record

parameters:

student (str): the name of the student to mark as absent

-- More --

Help on module attendance:

NAME

attendance

FUNCTIONS

get_attendance(student)

Returns the attendance status of a student.

parameters:

student (str): the name of the student whose attendance is to be retrieved

returns:

str: the attendance status of the student (Present or Absent)

mark_absent(student)

marks a student as absent in the attendance record

parameters:

student (str): the name of the student to mark as absent

mark_present(student)

marks a student as present in the attendance record

parameters:

student (str): the name of the student to mark as present

FILE

c:\users\souky\onedrive\documents\ai lab\attendance.py

attendance

[index](#)
[c:\users\admin\onedrive\desktop\aiac\attendance.py](#)

Attendance management module
this module provides functions to manage attendance records for students in a school or university setting.

Functions

```
get_attendance(student)
    Retrieves the attendance status of a student.
    Args:
        student (_type_): _description_
    Returns:
        _type_: _description_

mark_absent(student)
    Marks a student as absent in the attendance records.
    Args:
        student (_type_): _description_
    Returns:
        _type_: _description_

mark_present(student)
    Marks a student as present in the attendance records.
    Args:
        student (_type_): _description_
    Returns:
        _type_: _description_
```

Data

```
attendance_records = {}
```

Problem 5: File Handling Function

Consider the function:

```
def read_file(filename):
    with open(filename, 'r') as f:
        return f.read()
```

Task:

1. Write documentation using all three formats.
2. Identify which style best explains exception handling.
3. Justify your recommendation.

```

1  #DocString style:
2  def read_file(course):
3      """Reads the content of a file and returns it as a string.
4      Parameters:
5      file_name (str): The name of the file to be read.
6      Returns:
7      str: The content of the file as a string.
8      Raises:
9      FileNotFoundError: If the specified file does not exist.
10     IOError: If there is an error reading the file.
11     """
12
13     try:
14         with open(course, 'r') as f:
15             return f.read()
16     except FileNotFoundError:
17         print(f"Error: The file '{course}' was not found.")
18         raise
19     except IOError as e:
20         print(f"An I/O error occurred: {e}")
21         raise
22
23  #google style
24  def read_file(filename):
25      """
26      Reads the content of a file and returns it as a string.
27
28      Args:
29      filename (str): The name of the file to be read.

```

```

25     """
26     Reads the content of a file and returns it as a string.
27
28     Args:
29         filename (str): The name of the file to be read.
30
31     Returns:
32         str: The content of the file as a string.
33
34     Raises:
35         FileNotFoundError: If the specified file does not exist.
36         IOError: If there is an error reading the file.
37     """
38
39     try:
40         with open(filename, 'r') as f:
41             return f.read()
42     except FileNotFoundError:
43         print(f"Error: The file '{filename}' was not found.")
44     except IOError as e:
45         print(f"Error reading the file '{filename}': {e}")
46         raise
47
48 #Inline comments
49 def read_file(filename):
50     # open the file in read mode
51     # if the file does not exist, a FileNotFoundError will be raised
52     with open(filename, 'r') as f:
53         # read the content of the file and return it as a string
54         return f.read()
55
56 import course
57 help(course)

```

```

{'name': 'Introduction to Computer Science', 'credits': 4}
Course with ID HIST101 not found in the catalog.

This function takes a course ID as input and simulates retrieving course information from a course catalog.
Args:course_id (str): The unique identifier for the course to be retrieved.
Returns:
dict: A dictionary containing the course information if found, or a message indicating that the course was not found.

None
{'name': 'Introduction to Computer Science', 'credits': 4}
Course with ID HIST101 not found in the catalog.

This function takes a course ID as input and simulates retrieving course information from a course catalog.
Args:course_id (str): The unique identifier for the course to be retrieved.
Returns:
dict: A dictionary containing the course information if found, or a message indicating that the course was not found.

None

This function takes a course ID as input and simulates retrieving course information from a course catalog.
Args:course_id (str): The unique identifier for the course to be retrieved.
Returns:
dict: A dictionary containing the course information if found, or a message indicating that the course was not found.

This function takes a course ID as input and simulates retrieving course information from a course catalog.
Args:course_id (str): The unique identifier for the course to be retrieved.
Returns:
dict: A dictionary containing the course information if found, or a message indicating that the course was not found.

```